

File Uploads APIs

Complete documentation for file upload, management, and serving functionality.

Overview

The File Upload system provides:

- **Multi-format Support:** Images, documents, PDFs with intelligent processing
- **Cloud Storage:** Cloudinary integration for scalable file management
- **Security Features:** File type validation, virus scanning, and access controls
- **Image Processing:** Automatic resizing, compression, and optimization
- **Category Management:** Organized file storage by type and context

General File Uploads

POST /api/uploads/single

Upload a single file with automatic processing and cloud storage.

Authentication: Required

Headers:

```
Authorization: Bearer <access_token>
Content-Type: multipart/form-data
```

Request Body (multipart/form-data):

- `file` (file): Single file to upload

File Constraints:

- **Maximum file size:** 5MB
- **Allowed types:** JPEG, PNG, GIF, PDF, DOC, DOCX, TXT
- **Field name:** Must be exactly `file`
- **Security:** Automatic virus scanning and content validation

Example Request:

```
curl -X POST http://localhost:5000/api/uploads/single \
-H "Authorization: Bearer YOUR_ACCESS_TOKEN" \
-F "file=@document.pdf"
```

Response:

```
{
  "success": true,
  "message": "File uploaded successfully",
  "data": {
    "file": {
      "filename": "1647890123456-document.pdf",
      "originalName": "document.pdf",
      "mimetype": "application/pdf",
      "size": 2048576,
      "url": "https://cloudinary.com/v1_1/your-cloud/raw/upload/v1647890123/uploads/1647890123456-document.pdf",
      "publicId": "uploads/1647890123456-document",
      "category": "general",
      "uploadedAt": "2024-03-15T10:30:00.000Z",
      "dimensions": null,
      "metadata": {
        "pages": 5,
        "createdWith": "Adobe Acrobat"
      }
    }
  }
}
```

POST /api/uploads/multiple

Upload multiple files in a single request.

Authentication: Required

Headers:

```
Authorization: Bearer <access_token>
Content-Type: multipart/form-data
```

Request Body (multipart/form-data):

- **files** (files): Array of files to upload (max 5 files)

File Constraints:

- **Maximum files per request:** 5
- **Maximum file size:** 5MB each
- **Total request size:** 25MB maximum
- **Allowed types:** JPEG, PNG, GIF, PDF, DOC, DOCX

Example Request:

```
curl -X POST http://localhost:5000/api/uploads/multiple \
-H "Authorization: Bearer YOUR_ACCESS_TOKEN" \
-F "files=@image1.jpg" \
-F "files=@image2.png" \
-F "files=@document.pdf"
```

Response:

```
{
  "success": true,
  "message": "Files uploaded successfully",
  "data": {
    "files": [
      {
        "filename": "1647890123456-image1.jpg",
        "originalName": "image1.jpg",
        "url": "https://cloudinary.com/image1.jpg",
        "size": 1024000,
        "mimetype": "image/jpeg",
        "dimensions": {
          "width": 1920,
          "height": 1080
        }
      },
      {
        "filename": "1647890123457-image2.png",
        "originalName": "image2.png",
        "url": "https://cloudinary.com/image2.png",
        "size": 2048000,
        "mimetype": "image/png",
        "dimensions": {
          "width": 1280,
          "height": 720
        }
      },
      {
        "filename": "1647890123458-document.pdf",
        "originalName": "document.pdf",
        "url": "https://cloudinary.com/document.pdf",
        "size": 5120000,
        "mimetype": "application/pdf"
      }
    ],
    "uploadSummary": {
      "totalUploaded": 3,
      "totalSize": 8192000,
      "successfulUploads": 3,
      "failedUploads": 0
    }
  }
}
```

Event-Specific Uploads

POST /api/uploads/event-images

Upload images specifically for events with automatic categorization.

Authentication: Required (Vendor role)

Headers:

```
Authorization: Bearer <access_token>
Content-Type: multipart/form-data
```

Request Body (multipart/form-data):

- **images** (files): Event gallery images (max 10 files)
- **banner** (file): Event banner image (max 1 file)
- **thumbnail** (file): Event thumbnail image (max 1 file)

File Constraints:

- **Gallery images:** Max 10 files, 3MB each, JPEG/PNG only
- **Banner:** Max 1 file, 5MB, JPEG/PNG, recommended 1920x1080
- **Thumbnail:** Max 1 file, 2MB, JPEG/PNG, recommended 600x400
- **Auto-optimization:** Images are automatically compressed and optimized

Example Request:

```
curl -X POST http://localhost:5000/api/uploads/event-images \
-H "Authorization: Bearer YOUR_ACCESS_TOKEN" \
-F "banner=@event-banner.jpg" \
-F "thumbnail=@event-thumb.jpg" \
-F "images=@gallery1.jpg" \
-F "images=@gallery2.jpg" \
-F "images=@gallery3.jpg"
```

Response:

```
{
  "success": true,
  "message": "Event images uploaded successfully",
  "data": {
    "images": [
      "https://cloudinary.com/event-gallery-1.jpg",
      "https://cloudinary.com/event-gallery-2.jpg",
      "https://cloudinary.com/event-gallery-3.jpg"
    ],
    "banner": "https://cloudinary.com/event-banner-optimized.jpg",
    "thumbnail": "https://cloudinary.com/event-thumb-cropped.jpg",
    "uploadSummary": {
      "totalImages": 3,
      "hasBanner": true,
      "hasThumbnail": true,
      "totalSize": 8192000,
      "optimizationSavings": 2048000
    },
    "imageSpecs": {
      "banner": {
        "originalSize": 5120000,
        "optimizedSize": 3072000,
        "dimensions": "1920x1080",
        "format": "JPEG"
      },
      "thumbnail": {
        "originalSize": 1024000,
        "optimizedSize": 512000,
        "dimensions": "600x400",
        "format": "JPEG"
      }
    }
  }
}
```

Venue-Specific Uploads

POST /api/uploads/venue-images

Upload images and documents for venue listings.

Authentication: Required (Vendor role)

Headers:

```
Authorization: Bearer <access_token>
Content-Type: multipart/form-data
```

Request Body (multipart/form-data):

- **images** (files): Venue gallery images (max 20 files)
- **floorPlan** (file): Venue floor plan document (max 1 file)
- **thumbnail** (file): Venue thumbnail image (max 1 file)

File Constraints:

- **Gallery images:** Max 20 files, 3MB each, JPEG/PNG only
- **Floor plan:** Max 1 file, 10MB, JPEG/PNG/PDF formats

- **Thumbnail:** Max 1 file, 2MB, JPEG/PNG, recommended 600x400
- **Special processing:** Floor plans are optimized for viewing

Example Request:

```
curl -X POST http://localhost:5000/api/uploads/venue-images \
-H "Authorization: Bearer YOUR_ACCESS_TOKEN" \
-F "floorPlan=@venue-floor-plan.pdf" \
-F "thumbnail=@venue-main.jpg" \
-F "images=@venue-interior1.jpg" \
-F "images=@venue-interior2.jpg" \
-F "images=@venue-exterior.jpg"
```

Response:

```
{
  "success": true,
  "message": "Venue images uploaded successfully",
  "data": {
    "images": [
      "https://cloudinary.com/venue-interior1.jpg",
      "https://cloudinary.com/venue-interior2.jpg",
      "https://cloudinary.com/venue-exterior.jpg"
    ],
    "floorPlan": "https://cloudinary.com/venue-floor-plan.pdf",
    "thumbnail": "https://cloudinary.com/venue-thumbnail.jpg",
    "uploadSummary": {
      "totalImages": 3,
      "hasFloorPlan": true,
      "hasThumbnail": true,
      "totalSize": 15360000
    },
    "floorPlanInfo": {
      "format": "PDF",
      "pages": 1,
      "dimensions": "A3",
      "fileSize": 5120000
    }
  }
}
```

User Profile Uploads

POST /api/uploads/avatar

Upload user avatar with automatic processing and cropping.

Authentication: Required

Headers:

```
Authorization: Bearer <access_token>
Content-Type: multipart/form-data
```

Request Body (multipart/form-data):

- avatar (file): Avatar image file

File Constraints:

- **Maximum file size:** 2MB
- **Allowed types:** JPEG, PNG, GIF
- **Recommended size:** 300x300 pixels
- **Auto-processing:** Automatically cropped to square and optimized

Example Request:

```
curl -X POST http://localhost:5000/api/uploads/avatar \
-H "Authorization: Bearer YOUR_ACCESS_TOKEN" \
-F "avatar=@profile-picture.jpg"
```

Response:

```
{
  "success": true,
  "message": "Avatar uploaded successfully",
  "data": {
    "avatar": {
      "url": "https://cloudinary.com/avatars/user-123-avatar.jpg",
      "filename": "user-123-avatar.jpg",
      "size": 256000,
      "originalSize": 512000,
      "dimensions": {
        "width": 300,
        "height": 300
      },
      "format": "JPEG",
      "optimization": {
        "compressed": true,
        "quality": 85,
        "sizeSavings": 256000
      }
    },
    "user": {
      "_id": "user_123",
      "avatar": "https://cloudinary.com/avatars/user-123-avatar.jpg",
      "updatedAt": "2024-03-15T10:30:00.000Z"
    }
  }
}
```

Document Uploads

POST /api/uploads/document

Upload documents with security scanning and metadata extraction.

Authentication: Required

Headers:

```
Authorization: Bearer <access_token>
Content-Type: multipart/form-data
```

Request Body (multipart/form-data):

- **document (file):** Document file to upload

File Constraints:

- **Maximum file size:** 10MB
- **Allowed types:** PDF, DOC, DOCX, TXT, RTF
- **Security:** Automatic virus scanning and content validation
- **Processing:** Metadata extraction and text indexing

Example Request:

```
curl -X POST http://localhost:5000/api/uploads/document \
-H "Authorization: Bearer YOUR_ACCESS_TOKEN" \
-F "document=@contract.pdf"
```

Response:

```
{
  "success": true,
  "message": "Document uploaded successfully",
  "data": {
    "document": {
      "filename": "document-1647890123456.pdf",
      "originalName": "contract.pdf",
      "url": "https://cloudinary.com/documents/document-1647890123456.pdf",
      "size": 5242880,
      "mimetype": "application/pdf",
      "category": "documents",
      "uploadedAt": "2024-03-15T10:30:00.000Z",
      "metadata": {
        "pages": 5,
        "author": "Legal Department",
        "title": "Service Agreement Contract",
        "createdDate": "2024-03-10T00:00:00.000Z",
        "wordCount": 2500,
        "language": "en"
      },
      "security": {
        "virusScanned": true,
        "scanResult": "clean",
        "encrypted": false,
        "passwordProtected": false
      }
    }
  }
}
```

File Management & Serving

GET /api/uploads/files/:category/:filename

Serve uploaded files with proper headers and on-the-fly processing.

Authentication: Not required (for public files)

Path Parameters:

- `category` (string): File category ("users", "events", "venues", "documents")
- `filename` (string): File name to serve

Query Parameters:

- `download` (boolean): Force download instead of inline display ("true"/"false")
- `w` (number): Resize width for images (1-2000 pixels)
- `h` (number): Resize height for images (1-2000 pixels)
- `q` (number): Quality for images (1-100, default: 80)
- `format` (string): Convert image format ("jpg", "png", "webp")

Example Requests:

```
# Serve original file
curl http://localhost:5000/api/uploads/files/events/banner-123.jpg

# Resize image on-the-fly
curl "http://localhost:5000/api/uploads/files/events/banner-123.jpg?w=800&h=600&q=90"

# Force download
curl "http://localhost:5000/api/uploads/files/documents/contract.pdf?download=true"

# Convert format
curl "http://localhost:5000/api/uploads/files/events/image.png?format=webp&q=85"
```

Response:

- Returns file with appropriate Content-Type headers
- For images: Supports real-time resizing and format conversion
- For documents: Proper Content-Disposition headers for downloads

- Caching headers for optimal performance

GET /api/uploads/list/:category

List files in a specific category with filtering and pagination.

Authentication: Required

Headers:

```
Authorization: Bearer <access_token>
```

Path Parameters:

- `category` (string): File category to list

Query Parameters:

- `page` (number): Page number (default: 1)
- `limit` (number): Items per page (default: 20, max: 100)
- `type` (string): Filter by file type ("image", "document", "video")
- `sortBy` (string): Sort field ("uploadedAt", "size", "name", "mimetype")
- `sortOrder` (string): Sort order ("asc", "desc", default: "desc")
- `search` (string): Search by filename or original name
- `dateFrom` (string): Filter files uploaded after this date
- `dateTo` (string): Filter files uploaded before this date

Example Request:

```
curl -X GET "http://localhost:5000/api/uploads/list/events?type=image&limit=10&sortBy=uploadedAt" \
-H "Authorization: Bearer YOUR_ACCESS_TOKEN"
```

Response:

```
{
  "success": true,
  "message": "Files retrieved successfully",
  "data": {
    "files": [
      {
        "filename": "event-banner-123.jpg",
        "originalName": "summer-festival-banner.jpg",
        "size": 2048576,
        "mimetype": "image/jpeg",
        "url": "https://cloudinary.com/event-banner-123.jpg",
        "category": "events",
        "uploadedAt": "2024-03-15T10:30:00.000Z",
        "uploadedBy": {
          "_id": "vendor_123",
          "firstName": "Ahmed",
          "lastName": "Events"
        },
        "dimensions": {
          "width": 1920,
          "height": 1080
        },
        "fileType": "image"
      }
    ],
    "pagination": {
      "currentPage": 1,
      "totalPages": 5,
      "totalFiles": 47,
      "hasNextPage": true,
      "hasPrevPage": false,
      "limit": 10
    },
    "summary": {
      "totalFiles": 47,
      "totalSize": 104857600,
      "breakdown": {
        "images": 32,
        "documents": 15
      },
      "averageFileSize": 2230950
    }
  }
}
```

GET /api/uploads/info/:filename

Get detailed information about a specific file.

Authentication: Required

Headers:

Authorization: Bearer <access_token>

Path Parameters:

- `filename` (string): File name to get information for

Response:

```
{
  "success": true,
  "message": "File information retrieved successfully",
  "data": {
    "file": {
      "filename": "event-banner-123.jpg",
      "originalName": "summer-festival-banner.jpg",
      "mimetype": "image/jpeg",
      "size": 2048576,
      "url": "https://cloudinary.com/event-banner-123.jpg",
      "publicId": "events/event-banner-123",
      "category": "events",
      "uploadedAt": "2024-03-15T10:30:00.000Z",
      "uploadedBy": {
        "_id": "vendor_123",
        "firstName": "Ahmed",
        "lastName": "Events"
      },
      "dimensions": {
        "width": 1920,
        "height": 1080,
        "aspectRatio": 1.78
      },
      "metadata": {
        "colorSpace": "sRGB",
        "hasAlpha": false,
        "format": "JPEG",
        "quality": 85
      },
      "usage": {
        "usedInEvents": ["event_456", "event_789"],
        "totalViews": 1250,
        "lastAccessed": "2024-03-16T14:30:00.000Z"
      }
    }
  }
}
```

DELETE /api/uploads/file/:filename

Delete an uploaded file and remove it from cloud storage.

Authentication: Required (File owner or Admin)

Headers:

```
Authorization: Bearer <access_token>
```

Path Parameters:

- `filename` (string): File name to delete

Query Parameters:

- `category` (string): File category (required for validation)
- `force` (boolean): Force delete even if file is in use ("true"/"false")

Example Request:

```
curl -X DELETE "http://localhost:5000/api/uploads/file/old-banner-123.jpg?category=events" \
-H "Authorization: Bearer YOUR_ACCESS_TOKEN"
```

Response:

```
{
  "success": true,
  "message": "File deleted successfully",
  "data": {
    "deletedFile": "old-banner-123.jpg",
    "category": "events",
    "deletedAt": "2024-03-16T15:00:00.000Z",
    "cloudinaryResult": {
      "result": "ok",
      "publicId": "events/old-banner-123"
    }
  }
}
```

✖ Error Handling

Common Upload Errors

File Size Exceeded:

```
{
  "success": false,
  "message": "File size exceeds maximum allowed limit",
  "error": {
    "code": "FILE_SIZE_EXCEEDED",
    "details": "File size 6MB exceeds maximum allowed size of 5MB",
    "maxSizeBytes": 5242880,
    "actualSizeBytes": 6291456
  }
}
```

Invalid File Type:

```
{
  "success": false,
  "message": "File type not allowed",
  "error": {
    "code": "INVALID_FILE_TYPE",
    "details": "Only JPEG, PNG, GIF files are allowed for this endpoint",
    "allowedTypes": ["image/jpeg", "image/png", "image/gif"],
    "receivedType": "application/octet-stream"
  }
}
```

Too Many Files:

```
{
  "success": false,
  "message": "File count exceeds maximum allowed",
  "error": {
    "code": "FILE_COUNT_EXCEEDED",
    "details": "Maximum 5 files allowed per request, received 8",
    "maxFiles": 5,
    "receivedFiles": 8
  }
}
```

Virus Detected:

```
{
  "success": false,
  "message": "File contains malicious content",
  "error": {
    "code": "VIRUS_DETECTED",
    "details": "File failed security scan and has been quarantined",
    "scanResult": "threat_detected",
    "threatType": "malware"
  }
}
```

Storage Quota Exceeded:

```
{
  "success": false,
  "message": "Storage quota exceeded",
  "error": {
    "code": "QUOTA_EXCEEDED",
    "details": "User storage quota exceeded. Please delete some files or upgrade plan",
    "currentUsage": 104857600,
    "quotaLimit": 107374182,
    "quotaType": "user"
  }
}
```

Implementation Examples

File Upload with Progress Tracking

```
// React component for file upload with progress
const FileUploadComponent = ({ onUpload, maxFiles = 5 }) => {
  const [files, setFiles] = useState([]);
  const [uploading, setUploading] = useState(false);
  const [progress, setProgress] = useState({});

  const handleFileUpload = async (selectedFiles) => {
    setUploading(true);
    const formData = new FormData();

    Array.from(selectedFiles).forEach(file => {
      formData.append('files', file);
    });

    try {
      const response = await fetch('/api/uploads/multiple', {
        method: 'POST',
        headers: {
          'Authorization': `Bearer ${accessToken}`
        },
        body: formData,
        onUploadProgress: (progressEvent) => {
          const percentCompleted = Math.round(
            (progressEvent.loaded * 100) / progressEvent.total
          );
          setProgress(prev => ({ ...prev, overall: percentCompleted }));
        }
      });
    } catch (error) {
      console.error('Upload error:', error);
    } finally {
      setUploading(false);
      setProgress({});
    }
  };

  const result = await response.json();

  if (result.success) {
    onUpload(result.data.files);
    setFiles([]);
  } else {
    console.error('Upload failed:', result.message);
  }

  const validateFiles = (selectedFiles) => {
    const validFiles = [];
    const errors = [];

    Array.from(selectedFiles).forEach(file => {
```

```

// Size check
if (file.size > 5 * 1024 * 1024) {
  errors.push(`${file.name}: File too large (max 5MB)`);
  return;
}

// Type check
const allowedTypes = ['image/jpeg', 'image/png', 'image/gif', 'application/pdf'];
if (!allowedTypes.includes(file.type)) {
  errors.push(`${file.name}: File type not allowed`);
  return;
}

validFiles.push(file);
});

return { validFiles, errors };
};

return (
  <div className="file-upload">
    <input
      type="file"
      multiple
      accept="image/*,application/pdf"
      onChange={ (e) => {
        const { validFiles, errors } = validateFiles(e.target.files);
        if (errors.length > 0) {
          alert(errors.join('\n'));
          return;
        }
        handleFileUpload(validFiles);
      }}
      disabled={uploading}
    />

    {uploading && (
      <div className="upload-progress">
        <div className="progress-bar">
          <div
            className="progress-fill"
            style={{ width: `${progress.overall || 0}%` }}
          />
        </div>
        <p>Uploading... {progress.overall || 0}%</p>
      </div>
    )}
  </div>
);
};

```

Image Optimization Utility

```
// Utility for working with uploaded images
const ImageUtils = {
  // Generate responsive image URLs
  getResponsiveUrls: (baseUrl) => {
    const breakpoints = [320, 640, 768, 1024, 1280, 1920];

    return breakpoints.reduce((urls, width) => {
      urls[`w${width}`] = `${baseUrl}?w=${width}&q=80&format=webp`;
      return urls;
    }, {
      original: baseUrl
    });
  },

  // Generate srcset for responsive images
  generateSrcSet: (baseUrl) => {
    const responsive = ImageUtils.getResponsiveUrls(baseUrl);

    return Object.entries(responsive)
      .filter(([key]) => key !== 'original')
      .map(([key, url]) => `${url} ${key.substring(1)}w`)
      .join(', ');
  },

  // Create thumbnail URL
  getThumbnail: (baseUrl, size = 150) => {
    return `${baseUrl}?w=${size}&h=${size}&c=fill&q=80&format=webp`;
  },

  // Preload critical images
  preloadImage: (url) => {
    return new Promise((resolve, reject) => {
      const img = new Image();
      img.onload = () => resolve(img);
      img.onerror = reject;
      img.src = url;
    });
  }
};
```

File Manager Component

```
// File management interface
const FileManager = ({ category, userRole }) => {
  const [files, setFiles] = useState([]);
  const [loading, setLoading] = useState(false);
  const [pagination, setPagination] = useState({});
  const [filter, setFilter] = useState({ type: 'all', sortBy: 'uploadedAt' });

  useEffect(() => {
    fetchFiles();
  }, [category, filter]);

  const fetchFiles = async (page = 1) => {
    setLoading(true);
    try {
      const params = new URLSearchParams({
        page,
        limit: 20,
        ...filter
      });

      const response = await fetch(`/api/uploads/list/${category}?${params}`, {
        headers: { 'Authorization': `Bearer ${accessToken}` }
      });

      const result = await response.json();

      if (result.success) {
```

```

        setFiles(result.data.files);
        setPagination(result.data.pagination);
    }
} catch (error) {
    console.error('Failed to fetch files:', error);
} finally {
    setLoading(false);
}
};

const deleteFile = async (filename) => {
    if (!confirm('Are you sure you want to delete this file?')) return;

    try {
        const response = await fetch(`/api/uploads/file/${filename}?category=${category}`, {
            method: 'DELETE',
            headers: { 'Authorization': `Bearer ${accessToken}` }
        });

        const result = await response.json();

        if (result.success) {
            setFiles(prev => prev.filter(file => file.filename !== filename));
        }
    } catch (error) {
        console.error('Failed to delete file:', error);
    }
};

const formatFileSize = (bytes) => {
    const sizes = ['Bytes', 'KB', 'MB', 'GB'];
    if (bytes === 0) return '0 Bytes';
    const i = parseInt(Math.floor(Math.log(bytes) / Math.log(1024)));
    return Math.round(bytes / Math.pow(1024, i) * 100) / 100 + ' ' + sizes[i];
};

return (
    <div className="file-manager">
        <div className="file-controls">
            <select
                value={filter.type}
                onChange={(e) => setFilter(prev => ({ ...prev, type: e.target.value })))
            >
                <option value="all">All Files</option>
                <option value="image">Images</option>
                <option value="document">Documents</option>
            </select>

            <select
                value={filter.sortBy}
                onChange={(e) => setFilter(prev => ({ ...prev, sortBy: e.target.value })))
            >
                <option value="uploadedAt">Upload Date</option>
                <option value="size">File Size</option>
                <option value="name">Name</option>
            </select>
        </div>

        <div className="file-grid">
            {files.map(file => (
                <div key={file.filename} className="file-card">
                    {file.fileType === 'image' ? (
                        <img
                            src={`/${file.url}?w=200&h=200&c=fill&q=80`}
                            alt={file.originalName}
                            className="file-thumbnail"
                        />
                    ) : (
                        <div className="file-icon">📄</div>
                    )}
                </div>
            )}
        </div>
    </div>

```

```

    })

    <div className="file-info">
      <h4 title={file.originalName}>{file.originalName}</h4>
      <p>{formatFileSize(file.size)}</p>
      <p>{new Date(file.uploadedAt).toLocaleDateString()}</p>
    </div>

    <div className="file-actions">
      <a href={file.url} target="_blank" rel="noopener noreferrer">
        View
      </a>
      {userRole === 'admin' && (
        <button onClick={() => deleteFile(file.filename)}>
          Delete
        </button>
      )}
    </div>
  </div>
  )))
</div>

{pagination.totalPages > 1 && (
  <div className="pagination">
    {Array.from({ length: pagination.totalPages }, (_, i) => i + 1).map(page => (
      <button
        key={page}
        onClick={() => fetchFiles(page)}
        className={page === pagination.currentPage ? 'active' : ''}
      >
        {page}
      </button>
    ))}
  </div>
)}
</div>
);
};

```

Related Documentation:

- [Events → \(events.md\)](#) - Event image and banner management
- [Venues & Vendors → \(venues-vendors.md\)](#) - Venue images and vendor profile uploads
- [Authentication → \(./authentication.md\)](#) - User authentication for file access
- [Admin APIs → \(admin-apis.md\)](#) - Administrative file management functions